

Building an Image Recognition System for Quality Control

Business Report — Automated Defect Detection for Manufacturing QC

Executive Summary

Manual visual inspection of manufactured products does not scale well: inspectors get tired, different people make inconsistent calls, and defects can be small enough to miss. This project builds an automated image classification system that looks at a photo of a manufactured part and decides whether it is acceptable or defective, using machine learning trained on real casting product images. The goal is to support (not fully replace) human quality inspectors by flagging likely defects consistently and instantly, at any production volume.

1. Business Problem

Manufacturing companies inspect every product before it reaches a customer. As production volume grows, manual inspection becomes a bottleneck and a source of inconsistency:

- Thousands of units are produced daily, more than inspectors can carefully examine.
- Some defects are visually subtle and easy to overlook.
- Fatigue causes inspection quality to degrade over a shift.
- Different inspectors apply different standards, creating inconsistent outcomes.
- Fast-moving production lines leave little time per item for inspection.

2. Approach

We treat this as a binary image classification problem: given a photo of a part, predict **OK** (acceptable) or **Defective**. Two models were built and compared on the same held-out test set:

Model	Description	Why it's included
Baseline CNN	A convolutional neural network built and trained from scratch on the casting images.	Establishes a from-scratch performance baseline for comparison.
Transfer Learning (MobileNetV2)	A model pretrained on millions of general images (ImageNet), fine-tuned on our casting dataset.	Typically reaches higher accuracy faster on small/medium datasets like this one.

Both models were trained on the same augmented training data (with random rotations, shifts, zoom, and flips to reduce overfitting) and evaluated on an identical held-out test set that neither model saw during training.

3. How We Measured Success

Accuracy alone can be misleading in a quality-control setting, so the notebook reports several metrics side by side:

- **Accuracy** — overall percentage of correct predictions.
- **Precision** — of the items flagged as defective, how many truly are (avoids unnecessary rework).
- **Recall** — of the items that are truly defective, how many were caught (avoids shipping defects).
- **F1-score** — a balance between precision and recall.
- **ROC-AUC** — how well the model separates defective from acceptable items overall.

Note: Exact metric values depend on the training run (random initialization, number of epochs, hardware) and are reported in Section 9 of the accompanying notebook after you run it. This report focuses on how to interpret those numbers once you have them.

4. Interpreting the Results

In this use case, **recall on the defective class deserves the most attention**. A missed defect (false negative) means a bad product reaches a customer — a costly and reputation-damaging outcome. A false alarm (false positive) simply sends a good product for a second, unnecessary look — a much cheaper mistake. When comparing the two models in the notebook's results table, prioritize the model with higher recall on the defective class, even if it means slightly lower precision.

The confusion matrix and ROC curve in the notebook make this trade-off visible: a model with a few more false positives but almost no false negatives is generally preferable to a model that looks accurate overall but occasionally misses real defects.

5. Deployment Recommendations

- **Tune the decision threshold.** The model outputs a probability, not just a label. Lowering the cutoff below the default 0.5 catches more true defects at the cost of a few more false alarms — usually the right trade-off for QC.
- **Route uncertain cases to a human.** When the model's confidence is close to 50/50, send that item for manual review instead of an automatic decision.
- **Pilot before full rollout.** Run the model alongside existing inspectors for a trial period and compare decisions before removing manual review.
- **Monitor for drift.** Retrain periodically as new defect types, lighting conditions, or camera setups appear on the line — model performance degrades if the live data drifts too far from the training data.
- **Log every prediction.** Keep a record of model decisions and outcomes to support ongoing evaluation and continuous improvement.

6. Limitations

- Trained on a single product type (submersible pump impellers); performance on other product lines would need separate validation.
- Assumes a fairly consistent camera angle and lighting, matching the training images — significant changes to the physical inspection setup could reduce accuracy.
- This is an academic/demonstration project; a production deployment would need further testing, a larger and more diverse dataset, and integration with the actual production-line camera system.

7. Conclusion

This project demonstrates that an automated image classification system can support manufacturing quality control by consistently and instantly flagging likely defects. Transfer learning is expected to outperform a from-scratch CNN on this dataset size, and the full technical comparison, training curves,

and error analysis are available in the accompanying Jupyter notebook.

Prepared as part of AIML coursework — Task 13: Building an Image Recognition System for Quality Control